2520090016
P.Rahul

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

a) Write a C code to create a process using fork() system call.

b) Read the user input as shell command.

c) Use a system call exec() to execute the shell command.

e) Display the PIDs of parent and child processes.

```
root@Rahul:~# nano week1.c
root@Rahul:~#
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        printf("\n--- Child Process ---\n");
        printf("Child PID  : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("\nExecuting command: %s\n\n", command);

        execlp(command, command, NULL);

        perror("Command execution failed");
        exit(1);
    }
    else {
        printf("\n--- Parent Process ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

1)Then press ctrl+oenter ctrl+x2)then gcc filename.c -o filename

```
root@Rahul:~# nano week1.c
root@Rahul:~# gcc week1.c -o week1
```

3)then ./filename

```
root@Rahul:~# nano week1.c
root@Rahul:~# gcc week1.c -o week1
root@Rahul:~# ./week1
Enter a Linux command: ls
```

4)enter the linux command

```
root@Rahul:~# ./week1
Enter a Linux command: ls

--- Parent Process ---
Parent PID : 679

Child PID  : 680
--- Child Process ---
Child PID  : 680
Parent PID : 679

Executing command: ls

OSSP             command_executor.c  cpro.c  file.c       filename    process    task     week1    week3
command_executor  cpro               file    file.c.save  filename.c  process.c  task1.c  week1.c  week3.c

Child process completed.
```

Child process completed.

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Aim

To investigate the relationship between hardware resources and operating system services using Linux commands (uname, lscpu, ps, and top) and understand how the operating system abstracts CPU, memory, storage, and I/O devices.

Software Requirement

- Ubuntu Linux
- Terminal

Commands Used

1. ps
2. top

Screenshots

ps Command:

Purpose: Displays active processes with PID, TTY, TIME and CMD.

```
root@Rahul:~# ps
    PID TTY          TIME CMD
   2854 pts/2    00:00:00 bash
   3507 pts/2    00:00:00 ps
```

top Command:

Purpose: Displays real-time CPU, memory and process usage.

```
root@Rahul:~# top
top - 04:18:16 up  5:17,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  26 total,   1 running,  25 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7612.6 total,   6836.1 free,    581.3 used,    349.7 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7031.4 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   21724  13084   9728 S   0.0   0.2   0:02.59 systemd
      2 root      20   0    3180   2204   2072 S   0.0   0.0   0:00.07 init-systemd(Ub
      6 root      20   0    3196   2132   2008 S   0.0   0.0   0:00.00 init
     55 root      19  -1   33968  13272  12184 S   0.0   0.2   0:01.96 systemd-journal
    104 root      20   0   25008   6412   5148 S   0.0   0.1   0:04.94 systemd-udevd
    114 systemd+  20   0   21460  13276  11016 S   0.0   0.2   0:00.36 systemd-resolve
    118 systemd+  20   0   91028   7996   7024 S   0.0   0.1   0:00.92 systemd-timesyn
    182 root      20   0    4244   2684   2432 S   0.0   0.0   0:00.07 cron
    183 message+  20   0    9592   5428   4708 S   0.0   0.1   0:00.71 dbus-daemon
    198 root      20   0   17968   8836   7796 S   0.0   0.1   0:00.49 systemd-logind
    202 root      20   0 1756104  12576   8520 S   0.0   0.2   0:01.08 wsl-pro-service
    226 syslog    20   0  222516   5948   4596 S   0.0   0.1   0:00.45 rsyslogd
    228 root      20   0    3124   1976   1836 S   0.0   0.0   0:00.02 agetty
    246 root      20   0  107020  23204  13744 S   0.0   0.3   0:00.28 unattended-upgr
    326 root      20   0    6676   4508   3736 S   0.0   0.1   0:00.01 login
    369 root      20   0   20308  11576   9456 S   0.0   0.1   0:00.16 systemd
    372 root      20   0   21156   3496   1788 S   0.0   0.0   0:00.00 (sd-pam)
    394 root      20   0    6080   5304   3672 S   0.0   0.1   0:00.02 bash
    887 polkitd   20   0  308168   8036   7144 S   0.0   0.1   0:00.06 polkitd
   2848 root      20   0    3184   1116    980 S   0.0   0.0   0:00.00 SessionLeader
   2850 root      20   0    3200   1256   1108 S   0.0   0.0   0:00.13 Relay(2854)
   2854 root      20   0    6080   5364   3676 S   0.0   0.1   0:00.09 bash
   3250 root      20   0    3184   1116    980 S   0.0   0.0   0:00.00 SessionLeader
   3252 root      20   0    3200   1256   1108 S   0.0   0.0   0:00.01 Relay(3257)
   3257 root      20   0    6080   5384   3696 S   0.0   0.1   0:00.05 bash
   3508 root      20   0    9308   5744   3552 R   0.0   0.1   0:00.19 top
```

Hardware Resource Abstraction

CPU

The OS schedules processes and performs context switching.

Memory

The OS allocates RAM, provides virtual memory and protects processes.

Storage

The OS manages files using a file system and hides physical storage.

I/O Devices

The OS communicates with devices through device drivers.

Relationship Between Hardware Resources and OS Services

CPU → Process scheduling

Memory → Memory allocation and virtual memory

Storage → File system management

I/O Devices → Device drivers and interrupt handling

Result

The Linux commands were executed successfully. The experiment demonstrated how the operating system abstracts hardware resources and provides services for CPU, memory, storage and I/O devices**.**